

Survive the Crit — Design Review

Reviewer feedback, must-fix/nice-to-have sort, and evidence of fixes

Submitted the live portfolio (cevdetsatar-ml-portfolio.netlify.app) with the Chapter 1 proof statement — “I build AI systems the way a security audit would want them built” — to a reviewer, and asked the two opening questions before collecting the rest of the feedback.

The two opening questions

1. In ten seconds, what do I do?

“Yes, definitely. Your headline and the line right under it communicate the promise instantly. You didn't fall into the generic 'I build intelligent end-to-end ML solutions' trap most portfolios do — the topic is clear: enterprise AI systems that don't leak data and have passed adversarial testing.”

2. Would you believe I'm good at it?

“Halfway there — the potential is high, but right now the 'claim' side outweighs the 'proof' side. What convinced me: the 'isolation is enforced architecturally, not behaviorally' line, and the honest 'known limitation' box. What raised doubt: your claim is 'security audit,' but the entire Evaluation & Artifacts section — the load-bearing part of the site — said 'to be added' on every line. For an auditor, evidence you can't check is evidence that doesn't exist.”

Full feedback, round 1

1. “To be added” list undermines the section's own header

The section is titled “Check the work, don't take my word for it,” directly followed by three “to be added” lines — which defeats the header's own point.

2. Architecture diagram appeared missing/empty

The “Request flow” diagram under 2d showed as an empty area rather than rendering.

3. About section nearly duplicates the Case Study

Sentences in the About section were near-verbatim repeats of 02a/02c — a reader hits the same content twice instead of learning who the author is and how they think.

4. “18/18 scenarios passed” is a small claim for a security badge

3 accounts × 6 tests is only 6 distinct prompts — thin for anything read as a penetration test, even though the known-limitation box already discloses this honestly.

5. “Inert” explanation for the indirect-injection defense was ambiguous

Unclear whether the embedded instruction failed to execute by luck or by an explicit, structural rule — a real security reviewer would ask exactly this.

Full feedback, round 2 (after first fixes)

6. “Prototype source: not yet public — repo link coming” undercuts the fixed section

Right under a header about checking the work, an unfinished-looking placeholder was still the first line a reader saw.

7. Architectural vs. instruction-layer terminology needed to be more precise

The site calls isolation “architectural, not behavioral,” but the indirect-injection defense is implemented as a system-prompt rule — a security auditor could read that as an inconsistency unless the two defenses are explicitly named as separate layers for separate threats.

Closing verdict from the reviewer

“The first version felt like a big claim with an unfinished inside. The current version feels like solid, self-aware engineering work that knows its own limits. The claim now carries its own weight — you can put it forward with confidence.”

Must-fix / nice-to-have sort

Item	Sort	Why
1. 'To be added' eval artifacts	MUST-FIX	Directly contradicts the section's own claim; the proof doesn't land without it.
2. Missing architecture diagram	MUST-FIX	Broken/confusing — a security-focused site's most-expected visual wasn't rendering.
3. About duplicates Case Study	MUST-FIX	Confusing repetition; wastes the one section meant to show the person behind the work.
4. 18/18 statistical strength	NICE-TO-HAVE	Already honestly caveated in the limitation box; expanding to 180 requests needs new infrastructure and time.
5. 'Inert' explanation ambiguity	MUST-FIX	Goes to whether the core security claim is luck or design — hurts the one action if unresolved.
6. 'Not yet public' placeholder	MUST-FIX	Sits directly under the 'check the work' header; undercuts the section that was just fixed.
7. Architecture vs. instruction-layer wording	MUST-FIX	A precision gap a real security reviewer would flag — the proof needs to be airtight here.

What was changed on the live site

- Evaluation & Artifacts: replaced all three “to be added” lines with real, linked files — the adversarial test scripts and their result CSVs (English and original runs).
- Removed the unfinished-looking “Prototype source — not yet public” line entirely rather than leaving a placeholder, per the reviewer's own suggested fix.
- Re-deployed the architecture diagram (a stale deploy had been missing it) and verified it renders.
- Rewrote the About section from scratch — no sentence overlap with the Case Study — focused on personal philosophy (a statistics background and a habit of distrusting unverified numbers) rather than repeating project facts.

- Rewrote the indirect-injection explanation to name the defense explicitly as the instruction-layer half of a two-layer design (architectural isolation for cross-student data, instruction-layer separation for embedded document content), and named the concrete mechanism: the agreement text sits in its own labeled prompt section, separate from the rules section, with an explicit instruction never to treat that section as commands.

Not fixed (nice-to-have, deferred): expanding the adversarial suite to multiple phrasings per category (~180 requests) for a statistically stronger pass rate — already disclosed as a known limitation, and requires new test infrastructure rather than a same-day text fix.